

3.JS的加载与阻塞

一、JavaScript在渲染流程中的角色

JavaScript为网页带来了交互性，但它也是一把双刃剑。浏览器在解析HTML构建DOM的过程中，一旦遇到 `<script>` 标签，就会面临一个抉择。因为JavaScript有能力改变DOM的结构（例如使用 `document.write()` 或其他DOM操作API），所以浏览器必须谨慎行事。

二、默认 `<script>` 标签的阻塞行为

当HTML解析器遇到一个普通的 `<script>` 标签（没有 `async` 或 `defer` 属性）时，会发生以下情况：

1. **暂停DOM构建**: HTML解析器会立即停止解析页面的其余部分。
2. **下载脚本**: 浏览器会发出请求，下载该脚本文件（如果是外部脚本）。
3. **执行脚本**: 脚本下载完成后，JavaScript引擎会立即执行它。
4. **恢复DOM构建**: 脚本执行完毕后，HTML解析器才会继续解析剩余的HTML文档。

这个过程被称为“**解析器阻塞**” (**Parser Blocking**)。如果脚本下载耗时很长，或者执行时间过久，整个页面的渲染都会被“卡住”，导致用户长时间看到一个白屏。这就是为什么我们通常建议将 `<script>` 标签放在 `</body>` 标签之前的原因之一：确保浏览器能够先解析和渲染完整页面的主要内容。

三、优化手段：`async` 和 `defer`

为了解决脚本阻塞带来的性能问题，HTML5为 `<script>` 标签引入了两个布尔属性：`async` 和 `defer`。它们都告诉浏览器可以异步（在后台）下载脚本，而无需暂停DOM构建。

1. `async` (Asynchronous)

代码块

```
1 <script async src="path/to/script.js"></script>
```

- **行为:**
 - 脚本的下载过程与HTML解析并行进行（异步下载）。
 - 脚本下载完成后，HTML解析器会**立即暂停**，并执行该脚本。
 - 执行完毕后，恢复HTML解析。
- **执行顺序:** 多个带 `async` 的脚本，它们的执行顺序是**不确定**的。哪个脚本先下载完，哪个就先执行。
- **适用场景:** 适用于那些不依赖DOM、也不依赖其他脚本的独立脚本。例如，网站分析、广告脚本等。

2. `defer` (Deferred)

代码块

```
1 <script defer src="path/to/script.js"></script>
```

- **行为:**
 - 脚本的下载过程与HTML解析并行进行（异步下载）。
 - 脚本下载完成后，**并不会立即执行**。它会等待整个HTML文档解析完毕（即 `</html>` 标签被解析后），然后在 `DOMContentLoaded` 事件触发之前执行。
- **执行顺序:** 多个带 `defer` 的脚本，它们的执行顺序会按照它们在HTML中出现的顺序来**依次执行**。
- **适用场景:** 适用于那些需要操作DOM，或者脚本之间有依赖关系的场景。这是目前最推荐的脚本加载优化方案。

四、对比总结

属性	解析器阻塞	脚本下载	脚本执行时机	执行顺序
(无)	是	同步	下载后立即执行	按照在HTML中的顺序
<code>async</code>	否 (仅在执行时阻塞)	异步	下载后立即执行	不保证顺序
<code>defer</code>	否	异步	HTML解析完成后， <code>DOMContentLoaded</code> 之前	按照在HTML中的顺序

通过合理使用 `async` 和 `defer`，你可以显著减少JavaScript对关键渲染路径的阻塞，从而加快页面加载速度，提升用户体验。